Master 2 CIM - DA - DT 2026/2027
ICOM - Université de Lyon 2

Module Champ d'Application (Angular JS)

Travaux Pratiques 3

Composition de page et menus
Testé sur Angular version 22.1.8

Objectif: L'objectif de cette séance de TP est de revoir la génération de composants utiles pour structurer une page avec des menus.

Partie I: Génération de composants et styles SCSS

Dans ce TP, nous allons créer un nouveau projet différent des trois dernières séances et mettre en place la structure graphique d'une nouvelle application.

1.1 Créez un nouveau projet que vous nommerez `ModuleAngularTP3`. Pendant l'installation, choisissez le format de style `SCSS`.

1.2 Lancez le serveur et vérifiez que vous obtenez bien la même page de démarrage que pour les séances précédentes.

1.3 Créez deux nouveaux composants *home* et *about*.

Nouveauté : Nouveauté : les versions récentes d'*Angular* utilisent par défaut les composants *Standalone*. Les composants n'ont plus besoin d'être déclarés dans un *NgModule* : chaque composant peut importer directement les fonctionnalités dont il a besoin.

src > app > home > TS home.ts > ...

```ts
import { Component } from '@angular/core';

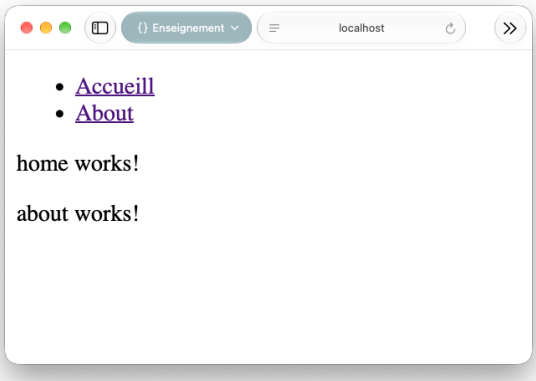
@Component({
  imports: [],
  selector: 'app-home',
  styleUrl: './home.scss',
  templateUrl: './home.html',
})
export class Home {}
```

1.4 Pour utiliser ces composants dans les composants de l'application racine, `App`, il est nécessaire de rajouter les deux composants dans les imports.

```ts
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Home } from './home/home';
import { About } from './about/about';

@Component({
  imports: [RouterOutlet, Home, About],
  selector: 'app-root',
  styleUrl: './app.scss',
  templateUrl: './app.html',
})
export class App {
  protected readonly title = signal('Tp3Correction');
}
```

1.5 Supprimez tout le contenu de la page par défaut du fichier `app.html` sauf la ligne `<router-outlet />`. Rajoutez une `unordered-list` avec un item défini comme un lien de type : `<a href="">Accueil</a>`. Faites un second item avec un second lien vers *About* (qui contiendra le lien "about"). Enfin, rajoutez une balise pour faire afficher par défaut votre composant app-home.
A noter : pour l'instant les liens ne fonctionneront pas mais vous devriez avoir une visualisation ressemblant à l'image ci-dessous.



1.6 Recopiez les styles css suivants dans leur fichiers respectifs. Ces styles sont accessibles à l'adresse suivante sur *moodle*. Vous devriez obtenir une page ressemblant à l'image suivante (a).

```scss
@import url('https://fonts.googleapis.com/css?family=Raleway:300,700');
body{
    background: #2E9CE6;
    padding: 3em;
    font-family: 'Raleway', 'Arial';
    color: #fff;
}

ul{
    list-style-type: none;
    margin: 0 0 2em 0;
    padding: 0;
}

ul li{
    display:inline;
    margin-right: 30px;
}

ul li {
    display: inline;
    margin-right: 30px;
}

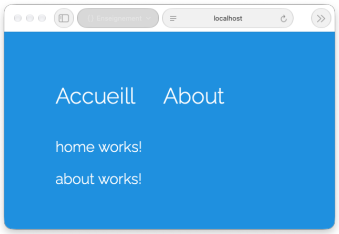
ul li a {
    font-size: 1.5em;
}
a{
    color: #fff;
    text-decoration: none;
}

```

`styles.scss`

```scss
.container {
    display: grid;
    grid-template-columns: 50% auto;
}
.col {
    padding: .4em 1.3em;
}
.color-dark {
    background: #2885C4;
}
.color-light {
    background: #57B3F1,
}
input.txt{
    border: 0;
    padding: 1em;
    width: 80%;
    margin-bottom: 2em;
}
input.btn{
    border: 0;
    display: block;
    padding: 1em 3em;
    background: rgba(230, 35, 10, 0.726);
    color: rgb(229, 239, 247);
    margin-bottom: 1em;
    cursor: pointer;
}
.life-container{
    background: #3FA0E1;
    padding: .5em;
    font-weight:bold;
    cursor:pointer;
}
```

`home.scss`



(a)

1.7 Utilisation de variables SCSS. Le fichier `styles.scss` contient plusieurs couleurs utilisées par l'application. SCSS permet de définir des variables afin d'éviter de répéter directement les valeurs des couleurs dans les règles CSS.

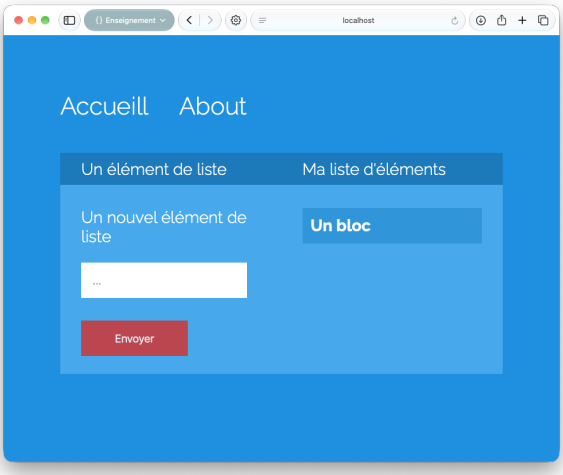
Dans le fichier `styles.scss`, définissez une variable `$main-color` contenant la couleur bleue utilisée pour le fond de la page :

```
$main-color: #2E9CE6;
```

Utilisez ensuite cette variable à la place de la valeur `#2E9CE6` dans la règle `body`.
Modifiez ensuite la variable pour observer directement le changement de couleur de la page.

À retenir : une variable SCSS commence par le caractère `$`. Elle peut être utilisée à plusieurs endroits dans les feuilles de style et sa valeur est remplacée lors de la compilation SCSS vers CSS.

1.8 Modifiez le code du fichier `home.html` de façon à obtenir la visualisation suivante :



Indications : le source doit contenir les éléments suivants :

- `<div class="container color-dark">`
- `<div class="container color-light">`
- `<div class="col">`
- `<p class="life-container">`
- Un formulaire contenant :
  - une balise input de type *text* de classe `txt`
  - une balise input de type submit de classe `btn`

```
<div class="container color-dark">
  <div class="col">
    <p> Premier bloc </p>
  </div>
  <div class="col">
      <p> Deuxième bloc </p>
  </div>
</div>
<div class="container color-light" >
    <div class="col">
      <p class="sm"> premier bloc dans containeer color-light </p>
    </div>
    <div class="col">
      <p class="life-container" >
        deuxième bloc dans container  color-light
      </p>
    </div>
</div>
```

Partie II: Ajout d'interactions

Maintenant que nous avons construit les base de l'interface, il s'agit de pouvoir rajouter dynamiquement des items depuis la zone de saisie vers la zone d'affichage (colonne de droite). Cette partie utilise uniquement les notions déjà vues dans les TP précédents.

2.1 Un premier élément dynamique de l'application est d'afficher le nombre de taches que l'utilisateur aura saisie dans l'interface. Pour cela dans le fichier `home.ts`, rajoutez une variable `nbItems` de type `number` initialisée à 4 et qui devra s'afficher à la suite du titre *Ma liste d'éléments* comme sur l'image suivante :



2.2 Comme pour la question précédente, on va rendre dynamique le texte du bouton du formulaire. Rajoutez une variable `btnText` de type `string` qui représentera le texte du bouton. Utilisez cette variable à l'aide d'un **property binding** pour afficher dynamiquement le texte du bouton.

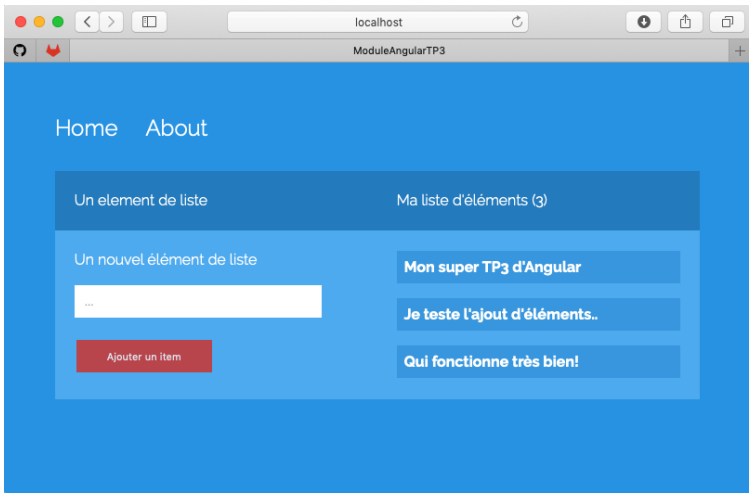
2.3 Pour importer `FormsModule` dans un projet *Standalone*, il faut désormais l'impoter directemeent dans le composant concerné (ici dans le `HomeComponent`). Importez la classe `FormsModule` définie dans `'@angular/forms'`. Vérifiez que l'import dans la section *imports* est bien effectuée automatiquement ou faites le manuellement. Dans le fichier `home.ts`, rajoutez une variable `objectifText` de type string qui sera utilisée dans la question suivante pour faire la double liaison.

2.4 Dans votre fichier `home.component.html`, utilisez la double liaison avec `[(ngModel)]` pour mettre à jour automatiquement votre variable précédente `objectifText` et affichez le contenu de cette variable dans un paragraphe de façon à obtenir le même résultat que pour l'image suivante.



2.5 Dans le fichier `home.component.ts`, rajoutez un tableau `objectifs` initialisé à un tableau vide, et dans la méthode `ngOnInit` initialisez correctement la variable précédente `nbItems`.

2.6 Toujours dans le même fichier, créez une nouvelle fonction `ajoutItem` qui rajoutera un nouvel élément au tableau (avec la méthode `push`) et qui mettra à jour la variable `nbItems`. Le nouvel élément rajouté sera défini à partir du texte écrit par l'utilisateur dans le formulaire.



2.7 Associez le bouton du formulaire à un événement de *click* de souris qui déclenchera l'appel à la fonction précédente (`ajoutItem`). Vérifiez que l'appel de la fonction est bien fonctionnel.

2.8 En utilisant un `@for`, modifiez la colonne du composant de classe `life-container` du fichier `home.component.html` de façon à afficher tous les éléments du tableau `objectifs`. Chaque élément sera de type : `<p class ="life-container"> element </p>`.